

Running the engine as a Windows service

Early Access · MessageFoundry v0.3.2 · September 2026

MessageFoundry runs as a background service through **NSSM**, the "Non-Sucking Service Manager". NSSM starts `messagefoundry serve` at boot, restarts it after a crash, and captures output in rotating logs. Stopping the service sends Ctrl+C so the engine drains its connections. The ASGI lifespan calls `engine.stop()`.

This guide covers a single-machine setup bound to `127.0.0.1`, with authentication required by default. For network access, set `[security].local_access_only = false` and `[security].listen_address`. Startup refuses an off-loopback API bind without TLS through `[api].tls_cert_file` or a trusted upstream terminator. See [DEPLOYMENT.md](#) for network controls and [REMOTE-CONSOLE.md](#) for remote operation.

Before going live, hand your endpoint-security and firewall admins the [Antivirus Exclusions & Firewall Permissions guide](#): it spells out the narrow set of AV exclusions (the SQLite store + `-wal` / `-shm` sidecars, logs, key/cert files, the venv interpreter) and the per-connection firewall openings the service needs.

Prerequisites

1. **Python venv with the package installed.** From the repo root: `powershell python -m venv .venv`
`.venv\Scripts\python.exe -m pip install -e .` This puts `messagefoundry.exe` in `.venv\Scripts\` — the service points at it. For a **reproducible, pinned** deployment, install the locked, hash-verified dependency set first, then the package itself: `powershell .venv\Scripts\python.exe -m pip install --require-hashes -r requirements.lock` `.venv\Scripts\python.exe -m pip install -e . --no-deps requirements.lock` is the SHA-256-pinned export checked in sync and audited in CI (DEP-1).
2. **NSSM** — provisioned automatically. If `nssm.exe` is not on `PATH` (or passed via `-NssmPath`), `install-service.ps1` downloads the pinned, SHA-256-verified release into `<DataDir>\bin\nssm.exe` and uses it. No manual install needed. (You can still pre-install it — `choco install nssm` or a download from <https://nssm.cc> — and it will be used if found.)
3. **An elevated PowerShell** (Run as Administrator) — required to register a service. `messagefoundry service install --env <name>` requests elevation through UAC. It runs the script in a visible window for output inspection.

Install

```
# from repo root, elevated PowerShell
.\scripts\service\install-service.ps1 -Environment prod
```

`-Environment` is **required** (ADR 0017): it selects which `environments/<name>.toml` value file the engine resolves and the instance's PHI posture. Without an environment, `serve` and the installer refuse to start. Select `dev`, `staging`, `prod`, or a custom name. For a custom name, set `[security].handles_real_patient_data` and `[security].production_instance` in `messagefoundry.toml`. Only `dev`, `staging`, and `prod` supply default PHI settings. Without both, `serve` exits 2 with *"environment '<name>' has no built-in security posture"*, so the service registers fine and then dies on every start. (The pre-ADR-0118 spellings `[ai].data_class` / `[ai].production` are **rejected at config load** — see ADR 0118.) `messagefoundry service install` requires the same name via `--env` and passes it straight through.

Defaults:

Setting	Default
Service name	MessageFoundry
Engine exe	<repo>\.venv\Scripts\messagefoundry.exe
Config dir	<repo>\samples\config
Active environment	(required — <code>-Environment</code>)
Data dir	C:\ProgramData\MessageFoundry
Message store	<DataDir>\messagefoundry.db
Logs	<DataDir>\logs\service.out.log , service.err.log
Bind	127.0.0.1:8765
Log level	INFO

Override any of them, e.g.:

```
.\scripts\service\install-service.ps1 -Environment prod -Port 9000 -LogLevel DEBUG `
-Config D:\hl7\config -DataDir D:\MessageFoundry
```

The install script is idempotent — re-running it reconfigures the existing service.

Migration note (ADR 0050, `--project-root`). `serve / supervise --project-root R` anchors the full configuration bundle under `R`. This includes `--config`, `environments/<env>.toml`, `messagefoundry.toml`, and relative `[store].path` or `--db` paths. Each shard's `<stem>_<shard>.db` also uses that root. Previously only `environments/` was anchored. A relative DB resolved against the process CWD. With `--project-root` and a relative database path, the database now resides under `R`. To retain its location, supply an absolute `[store].path` or `--db`. Absolute paths bypass the project root. The startup CWD-mismatch WARNING names the resolved paths so any move is visible. Deployments without `--project-root`, or with an absolute DB path, are unchanged.

Update to a new build (restart vs reinstall)

The service runs `<repo>\.venv\Scripts\messagefoundry.exe`. With an **editable** install (`pip install -e .`), it loads repository source at process start. After a pull, branch switch, or merge, **restart** the service from an elevated shell:

```
& C:\ProgramData\MessageFoundry\bin\nssm.exe restart MessageFoundry
curl http://127.0.0.1:8765/health
```

Because the install is editable, a restart runs **whatever branch is checked out** in the repo.

Reinstall instead when paths or flags change (port, config dir, data dir) or the service definition drifted — the install script is idempotent (it stops and reconfigures in place):

```
.\scripts\service\install-service.ps1 -Environment prod # elevated; re-points the exe + AppParameters
& C:\ProgramData\MessageFoundry\bin\nssm.exe start MessageFoundry
```

A non-editable install (`pip install .`) retains the previous code snapshot. Run `.venv\Scripts\python.exe -m pip install -e .`. Then restart the service.

⚠ Upgrading from $\leq 0.2.5 \rightarrow \geq 0.2.6$: tighten the config-dir ACLs first. The config-directory permission guard (SEC-003 / ADR 0036) did **not** exist in 0.2.5. From 0.2.6 on, `serve` refuses to start against a `--config` directory **writable by a broad principal** (e.g. `Authenticated Users / S-1-5-11`) — *"refusing to load config from writable-by-others path ..."*. A config dir that inherits that write (common under `C:\srv\...`) therefore **fails its first start after the upgrade**. Before the restart, restrict configuration-directory access. Use `Lock` down the config directory (CONFIG-2). The procedure uses `icacls /inheritance:d /T, /remove:g *S-1-5-11 /T`, and `SYSTEM/Admins` grants. It avoids a full `/inheritance:r` reset on a shared tree.

Start / stop / status

```
nssm start MessageFoundry
nssm status MessageFoundry
nssm stop MessageFoundry # Ctrl+C -> graceful connection shutdown (up to 15s)
nssm restart MessageFoundry
```

If `nssm` is not on `PATH`, it is the auto-downloaded copy at `<DataDir>\bin\nssm.exe` (e.g. `C:\ProgramData\MessageFoundry\bin\nssm.exe`). You can also use the built-in `sc.exe` / `Services.msc` once installed.

For desktop controls, run the **Windows tray service-manager** (`messagefoundry-tray`, included in every wheel installation). It shows status and provides start, stop, restart, console, and log access from the notification area.

Security hardening (recommended)

Run as a least-privilege account (DEPLOY-1)

The service defaults to the per-service virtual account `NT SERVICE\<ServiceName>`, such as `NT SERVICE\MessageFoundry`. This account needs **no password** (#224):

```
.\scripts\service\install-service.ps1 -Environment prod # runs as NT SERVICE\MessageFoundry
```

The engine needs **read** access to configuration and **read/write** access to data. The installer grants configuration read+execute and data read/write after registering the service, when the account SID resolves. Use the manual `icacls` commands below only for directories outside the managed paths. **LocalSystem** has broad local privileges. Choosing it increases the harm a compromised configuration module could cause. Enable it only through the explicit option:

```
.\scripts\service\install-service.ps1 -Environment prod -AllowLocalSystem # opt out to LocalSystem
```

To run under a **different** account (a domain gMSA, a dedicated local user, or another virtual account), pass `-ServiceAccount` — it always wins over the default:

```
.\scripts\service\install-service.ps1 -Environment prod -ServiceAccount "NT SERVICE\MessageFoundry"
```

```
# only if config/data live outside the script-managed paths:
icacls "D:\hl7\config" /grant "NT SERVICE\MessageFoundry:(OI)(CI)RX"
icacls "C:\ProgramData\MessageFoundry" /grant "NT SERVICE\MessageFoundry:(OI)(CI)M"
```

A domain **gMSA** or a dedicated local user works the same way (pass `-ServiceAccountPassword` for a password-based account — it is taken as a `SecureString`). The store file itself is further restricted to its owner at runtime. Account choice governs who that owner is.

If `-ServiceAccount` names a gMSA, the installer first runs `Test-ADServiceAccount` (#99). A gMSA name ends in `$`, for example `CORP\me-for-svc$`. This check confirms that the host can use the account. If necessary, run `Install-ADServiceAccount <name>`.

Before service registration, the installer uses `secedit` to grant `SeServiceLogonRight` ("Log on as a service"). Without that right, startup fails with **error 1069**. Both checks are best-effort and do not abort installation. On a non-domain host or a host without RSAT, the check logs a message and skips.

If a separate procedure prepares the account, use `-SkipGmsaPreflight`. Refer to [DEPLOY-SERVER-DB.md §1.1](#) for a SQL login and `[store].auth = "integrated"`.

Default flipped to least-privilege. `-AllowLocalSystem` is the LocalSystem opt-out (#224, built on #99).

Omitting both `-ServiceAccount` and `-AllowLocalSystem` now installs under the virtual account `NT SERVICE\<ServiceName>` — **not** LocalSystem. To run as LocalSystem you must pass `-AllowLocalSystem` explicitly (it prints a warning that LocalSystem is the acknowledged, non-default choice). An existing unattended install that already

passed `-ServiceAccount` is unaffected. An installation that requires the old LocalSystem behavior must specify `-AllowLocalSystem`. The recommended choice is the new virtual-account default. This flip is exercised end-to-end by the `windows-service-smoke` CI leg (a bare `-LockConfigDir` install, which now runs under the virtual account, must start and serve `/health` + MLLP on both Windows Server SKUs).

Protect the store encryption key at rest (WP-11d)

PHI columns are AES-256-GCM-encrypted at rest when a key is configured (see [PHI.md §3](#)). The key is a base64 32-byte secret. Two ways to supply it:

- **Environment (cross-platform default).** Set `MEFOR_STORE_ENCRYPTION_KEY` in the service's environment (`nssm set MessageFoundry AppEnvironmentExtra MEFOR_STORE_ENCRYPTION_KEY=...`). Simple, but the plaintext key sits in the service environment block, readable by any local administrator.
- **DPAPI-protected key file (Windows).** Use a DPAPI-protected file bound to this machine. A copied file is unusable elsewhere, and the environment contains no plaintext key:

```
powershell # mint + protect a fresh key (machine scope, so the service account can read it at
startup). # SYSTEM is granted read automatically (covers a LocalSystem service); for a virtual / gMSA
service # account add --grant-account '<that account>' so the service – not just you – can read the
key: messagefoundry protect-key --generate --out "C:\ProgramData\MessageFoundry\store.key.dpapi" #
(virtual account example: ... --grant-account "NT SERVICE\MessageFoundry") # -> prints the base64 key
ONCE to stderr; back it up offline (the file is machine-bound and # unrecoverable if the host is lost),
then point the engine at it: toml [store] encryption_key_file =
"C:/ProgramData/MessageFoundry/store.key.dpapi" Then unset MEFOR_STORE_ENCRYPTION_KEY (the env key takes
precedence when both are set). The service account CryptUnprotectDatas the file at startup. A
missing/foreign/unreadable file makes serve fail closed rather than store PHI unencrypted. protect-key grants
access to the creating administrator and read access to the selected service principal. The default service principal
is SYSTEM. For a virtual or gMSA account, use --grant-account. The file has an explicit DACL with inheritance
disabled. It does not inherit data-directory permissions. Grant the service account access when you create the key.
To rotate, protect-key a new key to the file and run messagefoundry rotate-key with the prior key in
MEFOR_STORE_ENCRYPTION_KEYS_RETIRED (see PHI.md §3).
```

External secrets manager. DPAPI provides local key protection. The engine reads environment variables and `encryption_key_file`. It does not contact a vault directly. For external secrets, use a service-start wrapper to retrieve the secret and set the applicable `MEFOR_*` variable. Sources can include Windows Credential Manager, HashiCorp Vault, Azure Key Vault with managed identity, or an AD gMSA for SQL/LDAP. Alternatively, use your provisioning tool to install the DPAPI key file. A direct broker remains future work.

Lock down the config directory (CONFIG-2)

`messagefoundry serve --config <dir>` and `POST /config/reload` **execute the Python** in the config directory in-process, with the service account's privileges. The directory is therefore a trust boundary: anyone who can write a `.py` file there can run code as the service.

- Restrict the config directory's ACL so only administrators / the service account can write it: `powershell icacls "D:\hl7\config" /inheritance:r /grant "Administrators:(OI)(CI)F" "NT SERVICE\MessageFoundry:(OI)(CI)R"` The supported one-step way to do this at install time is `install-service.ps1 -LockConfigDir`: It removes inherited ACEs. SYSTEM and Administrators receive full access. The service account receives read+execute access. No additional flag is necessary because the default service account is virtual. The option is explicit because configuration can reside in a developer repository. For production, select a dedicated administrator-owned directory with `-Config`. Then specify `-LockConfigDir`.
- **Fix an existing tree that inherits a broad write grant** without a full ACL reset. A config dir placed under a shared root (e.g. `C:\srv\...`) often inherits `Authenticated Users (S-1-5-11)` write, which trips the guard below. As an alternative to `/inheritance:r`, break inheritance. Remove the broad principal's grant. Give the service user read+execute access: `powershell icacls "C:\srv\mefor\config" /inheritance:d /T icacls "C:\srv\mefor\config" /remove:g *S-1-5-11 /T icacls "C:\srv\mefor\config" /grant "<run-as-user>:(OI)(CI)RX" /T`
- The loader **actively enforces** this at load time (and on `/config/reload`), not just as a documented recommendation (ADR 0036, SEC-003):
- On Windows, the loader checks the directory and each `*.py` file's NTFS owner and DACL. It rejects write-class grants to broad or low-privilege principals (Everyone, Authenticated Users, `BUILTIN\Users`, INTERACTIVE, ...). It also rejects such grants to any non-owner/non-administrator principal. Write-class rights include write, append, delete, `WRITE_DAC`, `WRITE_OWNER`, and generic-write. A `NULL` DACL (everyone allowed) is likewise refused. If a Win32 API error prevents DACL access, the guard allows loading with a WARNING. This preserves existing service operation. A warning about an unevaluated guard requires an ACL correction.
- On **POSIX** hosts the loader **refuses** to load from a group/world-writable or foreign-owned directory or module file.
- **Dev/test escape (never set in production)**. A default Windows checkout grants `BUILTIN\Users` write access. For an intentionally writable development/CI tree, set `MEFOR_ALLOW_INSECURE_CONFIG_SOURCE=1`. This changes the refusal to a WARNING. A production service leaves it unset and locks the config dir (above), so the guard stays fail-closed. The env var is the explicit, audited opt-out (mirrors `MEFOR_ALLOW_INSECURE_TLS`).
- `/config/reload` only loads from the startup `--config` directory and any directories listed in `[api].config_reload_roots` (see `CONFIGURATION.md`). An arbitrary path is rejected. Keep those roots admin-owned too.

Suppress Windows crash dumps of the engine (ADR 0152 Phase 0)

A Windows Error Reporting crash dump can write **PHI to disk**. Process memory holds HL7 bodies, decrypted data, and the unwrapped data-encryption key. Dump files sit outside the store's encryption, access controls, and retention sweep.

Each `serve` call sets process-local crash controls without configuration or privilege changes. It adds `SetErrorMode` flags without clearing inherited protections and calls `WerSetFlags(NOHEAP | NO_UI | DISABLE_SNAPSHOT_CRASH | DISABLE_SNAPSHOT_HANG)`. These calls do not change persistent machine settings.

Machine policy controls the remaining dump behavior. Windows checks `HKLM\...\Windows Error Reporting\LocalDumps` independently of WER exclusions and process flags. A host with LocalDumps configured can still create a dump. Apply the installation control below:

```
.\install-service.ps1 -Environment prod -SuppressCrashDumps
```

The option changes `HKLM` keys **by image name**, so it affects every process with that name on the host. Operators must select it explicitly.

It registers both `messagefoundry.exe` and the virtual environment's `python.exe`. The console-script launcher starts Python as a child process. Python holds the PHI in memory, so a rule for the launcher alone would not protect that memory.

LocalDumps is only ever narrowed, never switched on. WER local dump collection is **opt-in**: if the `LocalDumps` key does not exist, no local dumps are collected for anything on the host. Creating `LocalDumps\<image>` without an existing parent key would enable new per-image dump collection. The suppression option must not create that exposure. The installer writes per-image overrides only if a `LocalDumps` configuration already exists. It reports whether it changed that configuration. The override sets `DumpType=0` and `CustomDumpFlags=0` (`MiniDumpNormal`, without heap data). This stops full-dump configurations from collecting the engine's heap. Microsoft defines `DumpCount` as the maximum number of dump files in a folder. It does not disable dumps. Treat the override as a reduction: a `MiniDumpNormal` still carries thread stacks. To eliminate local dumps entirely, remove the host's `LocalDumps` configuration.

Limits remain. A postmortem debugger registered under `AeDebug` runs before WER and bypasses both controls. Registry keys remain after `uninstall-service.ps1`. Remove them manually to restore the host's original settings. Neither control has been tested by forcing a crash with PHI in memory and inspecting the dump directory. The Win32 calls succeeded, but the registry writes were not executed. These controls do not establish compliance with ASVS 11.7.1: memory hygiene does not encrypt memory.

Verify it's running

```
curl http://127.0.0.1:8765/health # -> {"status":"ok", ...}
```

Send a test message and confirm it flows through:

```
.\venv\Scripts\python.exe samples\send_mllp.py samples\messages\adt_a01.h17
```

Then check the log:

```
Get-Content C:\ProgramData\MessageFoundry\logs\service.out.log -Tail 20 -Wait
```

Startup should log unicorn's banner and `wiring started: N inbound, N outbound connection(s)`. A clean stop logs `wiring stopped` and `engine stopping`. A live configuration swap logs `wiring reloaded: ...`.

Logs

The engine uses Python `logging` for one timestamped UTC stdout/stderr stream. Refer to `messagefoundry/logging_setup.py`. It filters CR/LF log injection and applies `safe_exc()` PHI redaction to exceptions (WP-6c). Refer to [PHI.md §7](#).

NSSM captures these streams and rotates files at about 10 MB. For one JSON object per line, set `[logging].format = "json"`. For remote syslog/SIEM output, set `[logging].forward_host`. Select `forward_protocol = "tls"` for encrypted RFC 5425 transport. Set `forward_tls_ca_file` to the collector's PEM trust anchor.

Both outputs apply PHI redaction and control-character filtering. **Do not use `DEBUG` in production** because verbose output can include message content.

Restrict the log-directory ACL to administrators and the service account. Captured stdout/stderr contains operational data, rather than message bodies. Without this restriction, NSSM files inherit broadly readable `ProgramData` permissions (ASVS 16.4.2):

```
icaccls "C:\ProgramData\MessageFoundry\logs" /inheritance:r `
/grant "Administrators:(OI)(CI)F" "NT SERVICE\MessageFoundry:(OI)(CI)M"
```

Admin console (in a browser)

Operators use the **browser web console** at `/ui` to manage this background service. The engine mounts a separate, compatible console wheel in-process. The wheel is **not published to an index yet**, so this guide installs it by path:

```
pip install -e packaging/messagefoundry-webconsole # into the engine venv
```

No switch is needed: the console is **on by default** for a loopback bind ([ADR 0143](#)), and `[security].serve_web_console = false` turns it off. (`[api].serve_ui` was the old spelling and is now **refused at config load** — [ADR 0118](#) moved the console/bind/origin switches into `[security]`.)

Browse to this service's `/ui` (<http://127.0.0.1:8765/ui>) and sign in. See [INSTALL-GUIDE.md](#) → "Launching the admin console". (The former PySide6 desktop console was retired — [BACKLOG #103](#).)

High-delivery-rate TCP tuning (engine host)

Outbound MLLP defaults to one TCP connection per message (`persistent=false`) in this release ([ADR 0067 §8](#)). At hundreds of deliveries per second, the engine closes many connections. Their sockets remain in `TIME_WAIT` and can exhaust the default Windows ephemeral range of **16,384 ports**. Deliveries then dead-letter with `MLLP connect ... failed`, even when the partner is healthy.

In the 2026-07-02 load test, `TIME_WAIT` exceeded that range and about 50% of deliveries dead-lettered. The same load completed without exhaustion after the changes below. This concerns high-rate Pilot/Standard deployments. Most on-premises feeds do not reach those rates.

The optional `persistent=true` setting reuses one connection per destination (ADR 0067). This removes per-message handshakes and the associated `TIME_WAIT` buildup.

For sustained high-rate delivery, set `persistent = true` on the outbound `MLLP()` destination. A later release will change the default after the ADR 0067 §8 criterion passes. Until then, use persistent connections or the host settings below for high-rate feeds.

For hundreds of msg/s with `persistent=false`, use `persistent=true` on the outbound. Also consider this change for connect failures with reachable partners. Alternatively, widen the ephemeral port range and shorten the wait (administrator PowerShell on the engine host):

```
# Widen the ephemeral range (here: 22000-65535 ~= 43,500 ports)
netsh int ipv4 set dynamicport tcp start=22000 num=43535
# Halve how long a closed socket lingers (default 120s on older builds, 60s on newer)
Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters' `
  -Name TcpTimedWaitDelay -Value 30 -Type DWord # reboot to apply
```

Check settings with `netsh int ipv4 show dynamicport tcp` and `(Get-NetTCPConnection -State TimeWait).Count`. To restore defaults, set `start=49152 num=16384` and delete `TcpTimedWaitDelay`. Both changes affect the whole host. Coordinate with owners of other services.

Uninstall

```
.\scripts\service\uninstall-service.ps1
```

This stops and removes the service. The log files and message store under `DataDir` are left in place.

Troubleshooting

- **Service will not start / exits immediately.** Read `service.err.log`. The most common cause is a bad path baked into the service (relative paths resolve to the *system* directory for a service account). Re-run the install script, which resolves all paths to absolute.
- **Port already in use (e.g. 2575).** The sample config's inbound connection binds MLLP port `2575`. If a stray `messagefoundry serve` (or a second copy of the service) is already running, the listener fails to bind. Make sure only one instance runs: `Get-Process messagefoundry,python | Format-Table Id,ProcessName,Path`.
- **/health does not respond.** Confirm the service is `SERVICE_RUNNING` (`nssm status MessageFoundry`) and that nothing else owns port `8765`.
- **Permissions on the data dir.** The service runs by default as the least-privilege virtual account `NT SERVICE\<ServiceName>`, to which the installer grants read/write on `C:\ProgramData\MessageFoundry` (after registration, once the per-service SID resolves). If `-DataDir` is outside the managed path, grant the service account

read/write access there. The installer changes only the default directory's ACL. Without write access, startup fails. Choose a writable location and grant access. Alternatively, select LocalSystem with `-AllowLocalSystem` (see *Security hardening*).

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.